PS 3 — Controllers and Reinforcement Learning (Part A)

Sources: MathWorks, "What Is PID Control?" and Brian Douglas's "Understanding PID Control" videos (Gates 1–3). For Gate 4, the Stanley controller section of "Control Strategies for Autonomous Vehicles" (arxiv.org). For Gate 5, Hugging Face Deep RL Course, Unit 2 (huggingface.co).

Gate 1 — Feedback and error

Q1. What are a reference/setpoint, measured value, feedback and error?

Take the example of keeping a car in the centre of its lane.

- **Reference / setpoint**: the value you *want*. Here, 0 m from the lane centre.
- **Measured value**: the value you *actually have*, read from a sensor. Here, the camera says 0.4 m to the right.
- **Feedback**: feeding the measured value back to the controller so it can compare it with the setpoint, again and again.
- **Error**: setpoint − measured value. Here, 0 − 0.4 = −0.4 m. The sign tells which way to correct and the size tells how much.

Q2. Why is feedback usually better than blindly executing a precomputed command?

A precomputed command, called *open-loop* control, is like driving with your eyes shut by following a memorised plan such as "steer 5° for 3 seconds". Any surprise, such as wind, a slope, a slippery road or a slightly wrong model of the car, makes the vehicle drift, and it never notices. Feedback, called *closed-loop* control, keeps measuring and correcting, so small disturbances and modelling mistakes get fixed automatically. It is like steering a bicycle: you correct constantly rather than planning every movement in advance.

Gate 2 — P controller

Q1. Explain proportional control: u = Kp · e.

- **u** is the control action, for example the steering angle.
- **e** is the error.
- **Kp** is a number you choose, called the proportional gain.

The correction is *proportional* to the error. A big error gives a big correction and a small error gives a small one. Zero error gives no correction. If the car is 0.4 m off and Kp = 2, the controller commands 0.8 units of steering back toward the centre.

Q2. What happens when Kp is increased or decreased?

- **Higher Kp**: reacts strongly and reaches the setpoint faster. Too high and it overshoots, swings past the centre, corrects back, and oscillates (wobbles). It can even become unstable.
- **Lower Kp**: gentle and smooth, but slow. Too low and it barely corrects, so it lags behind and may never quite get there.

Tuning means finding a Kp that is fast without wobbling.

Q3. Why can a proportional controller still have steady-state error?

As the error shrinks, so does the correction. Suppose something constantly pushes the system, like a sidewind, a road tilt, or gravity on a hill. The system settles where the small P-correction exactly balances that push. At that point there is still a small error left, because zero error would mean zero correction, and the push would win again. That leftover error is the **steady-state error**. P alone cannot remove it. The I term below can.

Gate 3 — PID controller

Q1. Explain PID control.

› Source Σ

$$u(t) = K_p\, e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}$$

The control action is the sum of three terms, each with its own tuning gain (Kp, Ki, Kd):

- **P** uses the error right now.
- **I** uses the error added up over time. The ∫ symbol means "total so far". In code it is just integral += error * dt.
- **D** uses how fast the error is changing. In code it is (error - previous_error) / dt.

Q2. What is the intuitive role of P, I and D?

Term	Looks at	Intuition	Fixes	Side effect if too large
P	Present error	"I'm off by this much, correct by this much"	Most of the error, quickly	Overshoot and oscillation
I	Past error (accumulated)	"I've been slightly off for a while, push harder"	Steady-state error	Slow overshoot, wind-up

D	Future trend (rate of change)	"The error is closing fast, ease off before I overshoot"	Overshoot, adds damping	Jittery response to sensor noise

A car analogy: P is turning the wheel toward the centre. I notices you have always drifted slightly right, perhaps because of road camber, and adds a constant left trim. D notices you are swinging toward the centre quickly and straightens the wheel early so you do not swing past.

Gate 4 — Path tracking

Source: the Stanley controller section of "Control Strategies for Autonomous Vehicles" (arxiv.org), which defines both errors. Read only that section, since the rest is advanced.

Q1. What is cross-track error?

It is the **sideways distance** between the vehicle (usually its front axle) and the nearest point on the desired path, measured at right angles to the path. If the path is the lane centre and the car is 0.5 m to the left of it, the cross-track error is 0.5 m. It tells you *where you are* relative to the path.

Q2. What is heading error?

It is the **difference in angle** between the direction the vehicle is pointing and the direction of the path at that point. If the road runs straight north and the car points 10° east of north, the heading error is 10°. It tells you *which way you are going* relative to the path.

Q3. Why might both be important for smooth path following?

Each one alone misleads:

- **Only cross-track error:** the car might be exactly on the line but pointed sharply off it, and it will be off the line a moment later. It also tends to zig-zag across the path.
- **Only heading error:** the car might be parallel to the path but 2 m to the side, and it will stay 2 m off forever.

Using both means the controller corrects position (get back to the line) while also aligning the direction (point along the line). The car merges onto the path smoothly instead of overshooting. The Stanley controller combines exactly these two: steering = heading error + a term based on cross-track error and speed.

Gate 5 — Introduction to RL

Source: Hugging Face Deep RL Course, Unit 2 "Introduction to Q-Learning", and its hands-on FrozenLake notebook.

Q1. What are an agent, environment, state, action, reward and policy?

Take the FrozenLake game, which you will code in Part B. A character walks across a frozen lake on a 4 × 4 grid, trying to reach a goal square without falling into holes.

Term	Meaning	In FrozenLake
Agent	The learner and decision-maker	The character (your program)
Environment	Everything the agent interacts with	The lake grid with its holes and goal
State (s)	The current situation	Which of the 16 squares the agent is on
Action (a)	A choice the agent can make	Move left, down, right or up
Reward (r)	A number of feedback after each action	+1 for reaching the goal, 0 otherwise
Policy (π)	The agent's rule for choosing actions in each state	"On square 5, move right" for every square

The loop is: observe the state → choose an action → environment gives the new state and a reward → repeat. The goal of RL is to find the policy that collects the most total reward.

Q2. What does a Q-value Q(s, a) represent?

Q(s, a) is the agent's estimate of **how good it is to take action a in state s**. More precisely, it is the total future reward the agent expects if it takes that action now and then keeps behaving well afterwards.

- Q-learning stores these values in a table, the Q-table, with one row per state and one column per action. For FrozenLake that is 16 × 4.
- The table starts at all zeros. After each move, the agent nudges the value toward what it just observed, using the update rule:

› Source Σ

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

-]α (alpha) is the learning rate, meaning how much to trust the new experience. γ (gamma) is the discount factor, meaning how much future rewards matter compared with immediate ones. s' is the new state after the move.

- Once learned, the policy is simple: in each state, pick the action with the highest Q-value. During training the agent sometimes picks a random action instead, a strategy called ε-greedy, so it keeps exploring new possibilities.